

Cálculo Paralelo do Conjunto de Mandelbrot com MPI + OpenMP

Versão híbrida: MPI Coordenador/Trabalhador entre nós e *workpool* OpenMP (sem mestre) dentro do nó

Bernardo Luiz Padilha Zamin • João Pedro Sostruznik Sotero da Cunha
Programação Paralela — Trabalho 4 (MPI + OpenMP), PUCRS — Escola Politécnica, 2026

1. Introdução

Implementa-se, em C, a versão **híbrida MPI + OpenMP** do cálculo do **conjunto de Mandelbrot** (imagem 7680×4320), reaproveitando o *Coordenador/Trabalhador* do grupo. São **dois níveis de paralelismo**: o **MPI** distribui blocos de linhas entre os **nós (um processo pesado por nó)** e o **OpenMP** usa os núcleos *dentro* de cada nó no modelo **workpool sem mestre**. Avaliam-se *speed-up*, eficiência e escalabilidade **forte** em **4 nós**, o efeito do **Hyper-Threading**, as **formas de alocar o coordenador** e a **comparação com a MPI pura**.

2. Modelo híbrido

Entre nós (MPI). Modelo **Coordenador/Trabalhador idêntico** ao da MPI pura. O rank 0 distribui **blocos de rows_per_task linhas sob demanda**: o trabalhador pede (TAG_REQUEST), recebe TAG_TASK ou TAG_STOP, calcula e devolve (TAG_RESULT). Há **um processo pesado por nó**, com todos os núcleos à disposição das threads. Para cada pixel itera-se $z \leftarrow z^2 + c$ até $|z| > 2$ ou `max_iter`; pontos *internos* custam `max_iter` e *externos* terminam cedo, gerando **carga irregular** que exige **balanceamento dinâmico**.

3. Workpool OpenMP sem mestre

Cada bloco é processado por uma **região paralela** (`#pragma omp parallel`). O controle está numa **estrutura de dados compartilhada** `WorkPool{next,total}`: **todas as threads** executam o mesmo laço e **retiram atômica**mente a próxima linha (`#pragma omp atomic capture`) até esvaziar o pool — **sem thread mestre**, com escalonamento *self-service* emergindo da estrutura de dados. A granularidade é de **uma linha** (7680 px). Usa-se `MPI_Init_thread` (`MPI_THREAD_FUNNELED`) e `OMP_NUM_THREADS` de 1 a 16 (até 2 threads por núcleo, HT).

4. Alocação do coordenador

O coordenador só distribui/coleta, ficando **quase ocioso**. Adota-se **sempre** `-N 4 -n 5`: 4 nós, **1 coordenador + 4 trabalhadores**; o SLURM coloca o coordenador (rank 0) e um trabalhador (rank 1) *no mesmo nó* (confirmado: ambos em atlantica05). Comparam-se três alocações: (i) **dedicar 1 núcleo** (nó 0 usa 15 threads, 1 núcleo livre p/ o coordenador); (ii) **não dedicar** (nó 0 usa 16 threads, compartilhando); (iii) **dedicar um nó** (`-N 4 -n 4`, 3 trabalhadores).

5. Metodologia

4 nós da Atlantica (LAD), SLURM, Xeon E5520, **16 CPUs lógicas/nó** (8 núcleos $\times$ 2 HT). `ladcomp -env mpicc -O3 -fopenmp`. Tempo por `MPI_Wtime` antes do I/O. Baseline $T(1) = 93,19$ s (sequencial, `max_iter=2000`). O híbrido roda **sempre em** `-N 4 -n 5` e a escala se dá pelo **nº de threads por trabalhador** (`OMP_NUM_THREADS` $\in \{1, 2, 4, 8, 16\}$, ou seja **4 a 64 núcleos** de cálculo = 4 trab. $\times$ threads). Mantém-se `max_iter=2000` fixo. $\text{Speed-up} = T(1)/T(p)$; $\text{Eficiência} = \text{Speed-up}/p$, $p = 4 \times \text{threads}$. A **MPI pura** (1 processo por unidade, 1–64 processos) foi medida no seu caso de teste (`max_iter=1000`, $T(1) = 76,72$ s); compara-se pelas *métricas* (speed-up/eficiência). Saída **byte-idêntica** à sequencial (md5).

6. Resultados e análise

Escalabilidade forte (Tab. 1–2, Fig. a–b). Aumentando as unidades de cálculo, o híbrido (threads) e a MPI pura (processos) escalam quase idealmente, chegando a **46 $\times$** e **44 $\times$** em 64 unidades. O híbrido é *superlinear* com poucas unidades ($E > 1$): o menor *working-set* por núcleo melhora o uso de **cache**. A eficiência cai a $\sim 0,7$ em 64 unidades (Fig. b) por causa do **Hyper-Threading**: 64 unidades = 2 fluxos por núcleo físico, e o HT não dobra o desempenho.

Híbrido $\times$ MPI pura. As curvas (Fig. a, b) são **praticamente coincidentes**: neste problema *compute-bound* e independente, distribuir por *threads* ou por *processos* rende igual. A vantagem do híbrido é usar **muito menos ranks MPI** (5 vs 64 em 4 nós) — menos memória e mensagens, ganho que cresce com a escala.

Alocação do coordenador (Tab. 3, Fig. c). *Dedicar 1 núcleo* ao coordenador (nó 0 com 15 threads) é **$\approx 16\%$ mais rápido** que *não dedicar* (2,09 vs 2,48 s): com 16 threads + coordenador o nó 0 fica *oversubscrito* (17 fluxos em 16 CPUs), e reservar 1 núcleo elimina a contenção. Já *dedicar um nó inteiro* (n4) é o pior caso (**4,04 s**). Ou seja: compensa dedicar 1 núcleo, nunca um nó.

7. Conclusão

A versão cumpre o modelo (**MPI Coordenador/Trabalhador**, 1 processo/nó + **workpool OpenMP sem mestre**). Escalando as threads em `-N 4 -n 5`, atinge **46,3 $\times$** (64 núcleos), **empatado com a MPI pura**. Vale **dedicar 1 núcleo ao coordenador** ($\approx 16\%$ mais rápido), mas **nunca um nó** (n4: +63%). A queda de eficiência é o limite do Hyper-Threading, não do algoritmo.

Tabelas e gráficos

Tabela 1 — Híbrido (escalab. forte)

un.	Tempo (s)	S	E
4	21,89	4,26	1,06
8	10,98	8,49	1,06
16	5,86	15,9	0,99
32	3,31	28,1	0,88
64	2,01	46,3	0,72

Tabela 2 — MPI pura (escalab. forte)

un.	Tempo (s)	S	E
4	25,79	2,97	0,74
8	11,09	6,92	0,86
16	5,35	14,3	0,90
32	2,73	28,1	0,88
64	1,74	44,1	0,69

Tabela 3 — Alocação do coordenador (4 nós; lote controlado)

Alocação do coordenador	Tempo (s)	vs. não dedicar
<i>Dedica 1 núcleo</i> (n5, nó 0 = 15 th)	2,09	-16%
<i>Não dedica</i> (n5, nó 0 = 16 th)	2,48	—
<i>Dedica um nó</i> (n4, 3 trabalhadores)	4,04	+63%

un. = unidades de cálculo: no híbrido = núcleos ($4 \times \text{threads}$, $\text{max_iter}=2000$, $T(1) = 93,19$ s); na MPI pura = processos ($\text{max_iter}=1000$, $T(1) = 76,72$ s). Speed-up $S = T(1)/T(p)$; Eficiência $E = S/\text{un.}$; ideal = linear. Os **tempos absolutos não são comparáveis entre as versões** (max_iter difere): a comparação se faz pelas *métricas S e E*. Tab. 3 é um lote controlado à parte (comparar razões, não absolutos). Imagem 7680×4320 .

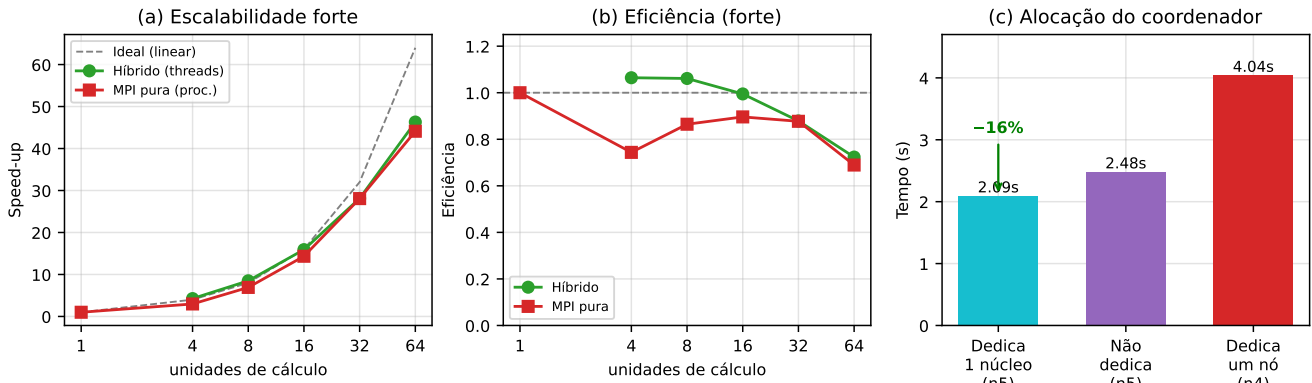


Figura 1 — híbrido $\times$ MPI pura: (a) escalabilidade forte (*speed-up*); (b) eficiência (forte); (c) alocação do coordenador.

Anexo — Código-fonte (mandelbrot-hib.c, trechos relevantes)

Versão híbrida MPI + OpenMP. O coordenador (rank 0) distribui blocos de linhas sob demanda e coleta os resultados (idêntico ao mandelbrot-mpi.c); cada trabalhador calcula seu bloco com o *workpool* OpenMP (compute_task_workpool, sem mestre). As funções inalteradas (mandelbrot, save_ppm) e o laço do coordenador são omitidos por brevidade. Compilação: `ladcomp -env mpicc -O3 mandelbrot-hib.c -o mandelbrot-hib -fopenmp -lm`.

```
1 #include <mpi.h>
2 #include <omp.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <math.h>
6 #define WIDTH 7680
7 #define HEIGHT 4320
8 #define DEFAULT_ROWS_PER_TASK 64
9 #define TAG_REQUEST 0
10 #define TAG_TASK 1
11 #define TAG_RESULT 2
12 #define TAG_STOP 3
13 typedef struct { int start_row; int num_rows; } Task;
14 /* Estrutura de dados compartilhada que CONTROLA o workpool (sem mestre):
15    next = índice da próxima linha do bloco ainda não tomada. */
16 typedef struct { int next; int total; } WorkPool;
17 int mandelbrot(double real, double imag, int max_iter); /* z=z^2+c; igual ao seq/MPI */
18 void save_ppm(const char *f, int *img, int max_iter); /* igual ao seq/MPI */
19 /* ===== WORKPOOL OpenMP: todas as threads trabalham, SEM MESTRE ===== */
20 void compute_task_workpool(const Task *task, int *buffer, int max_iter)
21 {
22     WorkPool pool = { .next = 0, .total = task->num_rows };
23     #pragma omp parallel /* todas as threads entram e trabalham */
24     {
25         while (1) {
26             int row;
27             #pragma omp atomic capture /* retira atomicamente a próxima linha */
28             { row = pool.next; pool.next++; } /* do pool compartilhado (sem mestre) */
29             if (row >= pool.total) break; /* pool vazio: esta thread termina */
30             int y = task->start_row + row;
31             for (int x = 0; x < WIDTH; x++) {
32                 double real = -2.5 + (3.5 * x / WIDTH);
33                 double imag = -1.0 + (2.0 * y / HEIGHT);
34                 buffer[row * WIDTH + x] = mandelbrot(real, imag, max_iter);
35             }
36         }
37     }
38 }
39 int main(int argc, char *argv[])
40 {
41     int max_iter = atoi(argv[1]);
42     int rows_per_task = (argc >= 3) ? atoi(argv[2]) : DEFAULT_ROWS_PER_TASK;
43     int provided; /* so a thread principal chama MPI */
44     MPI_Init_thread(&argc, &argv, MPI_THREAD_FUNNELED, &provided);
45     int rank, size;
46     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
47     MPI_Comm_size(MPI_COMM_WORLD, &size);
48     if (rank == 0) {
49         /* COORDENADOR: distribui blocos sob demanda (TAG_TASK) e coleta os
50            resultados (TAG_RESULT); cronometra (MPI_Wtime) e grava a imagem.
51            Idêntico ao mandelbrot-mpi.c (laço MPI_Probe / MPI_Recv / MPI_Send). */
52     } else {
53         /* TRABALHADOR: pede tarefa, calcula com o workpool, devolve. */
54         MPI_Status st; int request = 1;
55         while (1) {
56             MPI_Send(&request, 1, MPI_INT, 0, TAG_REQUEST, MPI_COMM_WORLD);
57             Task task;
58             MPI_Recv(&task, sizeof(Task), MPI_BYTE, 0, MPI_ANY_TAG,
59                    MPI_COMM_WORLD, &st);
60             if (st.MPI_TAG == TAG_STOP) break;
61             int *buffer = malloc(sizeof(int) * task.num_rows * WIDTH);
62             compute_task_workpool(&task, buffer, max_iter); /* <= OpenMP */
63             MPI_Send(&task, sizeof(Task), MPI_BYTE, 0, TAG_RESULT, MPI_COMM_WORLD);
64             MPI_Send(buffer, task.num_rows * WIDTH, MPI_INT, 0, TAG_RESULT,
65                    MPI_COMM_WORLD);
66             free(buffer);
67         }
68     }
69     MPI_Finalize();
70     return 0;
71 }
```

Anexo — Código-fonte (mandelbrot-mpi.c, versão MPI pura)

Versão MPI pura (Coordenador/Trabalhador). Base sobre a qual a versão híbrida foi construída (o coordenador e o protocolo de mensagens são idênticos; o híbrido apenas troca o laço de cálculo do trabalhador pelo *workpool* OpenMP). Compilação: `ladcomp -env mpicc -O3 mandelbrot-mpi.c -o mandelbrot-mpi -lm`.

```
1 #include <mpi.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <math.h>
5
6 #define WIDTH 7680
7 #define HEIGHT 4320
8
9 #define TAG_REQUEST 0
10 #define TAG_TASK 1
11 #define TAG_RESULT 2
12 #define TAG_STOP 3
13
14 typedef struct
15 {
16     int start_row;
17     int num_rows;
18 } Task;
19
20 int mandelbrot(double real, double imag, int max_iter)
21 {
22     double z_real = 0.0;
23     double z_imag = 0.0;
24
25     int iter = 0;
26
27     while ((z_real * z_real + z_imag * z_imag <= 4.0) &&
28           iter < max_iter)
29     {
30         double temp = z_real * z_real - z_imag * z_imag + real;
31
32         z_imag = 2.0 * z_real * z_imag + imag;
33         z_real = temp;
34
35         iter++;
36     }
37
38     return iter;
39 }
40
41 void save_ppm(const char *filename, int *image, int max_iter)
42 {
43     FILE *fp = fopen(filename, "w");
44
45     if (fp == NULL)
46     {
47         printf("Erro ao criar o arquivo %s\n", filename);
48         return;
49     }
50
51     fprintf(fp, "P3\n");
52     fprintf(fp, "%d %d\n", WIDTH, HEIGHT);
53     fprintf(fp, "255\n");
54
55     for (int i = 0; i < WIDTH * HEIGHT; i++)
56     {
57         int color = (image[i] * 255) / max_iter;
58
59         fprintf(fp, "%d %d %d ",
60               color,
61               color,
62               color);
63
64         if ((i + 1) % WIDTH == 0)
65             fprintf(fp, "\n");
66     }
67
68     fclose(fp);
69 }
70
71 int main(int argc, char *argv[])
72 {
73     if (argc != 2)
74     {
75         return 1;
76     }
77
78     int max_iter = atoi(argv[1]);
79
80     MPI_Init(&argc, &argv);
81
82     int rank, size;
83     double start, end;
84
85     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
86     MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```

87
88
89
90     if (rank == 0)
91         printf("Execute com pelo menos 2 processos: 1 coordenador e 1 trabalhador.\n");
92
93     MPI_Finalize();
94     return 0;
95 }
96
97 // =====
98 // COORDENADOR
99 // =====
100
101 if (rank == 0)
102 {
103     start = MPI_Wtime();
104
105     int *image = malloc(sizeof(int) * WIDTH * HEIGHT);
106
107     if (image == NULL)
108     {
109         printf("Erro ao alocar memoria para a imagem.\n");
110         MPI_Abort(MPI_COMM_WORLD, 1);
111     }
112
113     int next_row = 0;
114     int rows_completed = 0;
115     int rows_per_task = 10;
116     int workers_stopped = 0;
117
118     MPI_Status status;
119
120     /*
121      * Agora a iniciativa e dos trabalhadores:
122      * - o trabalhador envia TAG_REQUEST pedindo trabalho;
123      * - o coordenador responde com TAG_TASK ou TAG_STOP;
124      * - depois o trabalhador envia TAG_RESULT com o resultado.
125      */
126     while (workers_stopped < size - 1)
127     {
128         MPI_Probe(MPI_ANY_SOURCE,
129                 MPI_ANY_TAG,
130                 MPI_COMM_WORLD,
131                 &status);
132
133         int worker = status.MPI_SOURCE;
134
135         if (status.MPI_TAG == TAG_REQUEST)
136         {
137             int request;
138
139             MPI_Recv(&request,
140                     1,
141                     MPI_INT,
142                     worker,
143                     TAG_REQUEST,
144                     MPI_COMM_WORLD,
145                     MPI_STATUS_IGNORE);
146
147             if (next_row < HEIGHT)
148             {
149                 Task task;
150
151                 task.start_row = next_row;
152                 task.num_rows = rows_per_task;
153
154                 if (next_row + rows_per_task > HEIGHT)
155                     task.num_rows = HEIGHT - next_row;
156
157                 MPI_Send(&task,
158                         sizeof(Task),
159                         MPI_BYTE,
160                         worker,
161                         TAG_TASK,
162                         MPI_COMM_WORLD);
163
164                 next_row += task.num_rows;
165             }
166             else
167             {
168                 Task stop_task;
169                 stop_task.start_row = -1;
170                 stop_task.num_rows = 0;
171
172                 MPI_Send(&stop_task,
173                         sizeof(Task),
174                         MPI_BYTE,
175                         worker,
176                         TAG_STOP,
177                         MPI_COMM_WORLD);
178
179                 workers_stopped++;

```

```

180     }
181 }
182 else if (status.MPI_TAG == TAG_RESULT)
183 {
184     Task result_task;
185
186     MPI_Recv(&result_task,
187             sizeof(Task),
188             MPI_BYTE,
189             worker,
190             TAG_RESULT,
191             MPI_COMM_WORLD,
192             MPI_STATUS_IGNORE);
193
194     int pixels = result_task.num_rows * WIDTH;
195
196     MPI_Recv(&image[result_task.start_row * WIDTH],
197             pixels,
198             MPI_INT,
199             worker,
200             TAG_RESULT,
201             MPI_COMM_WORLD,
202             MPI_STATUS_IGNORE);
203
204     rows_completed += result_task.num_rows;
205 }
206 }
207
208 end = MPI_Wtime();
209
210 save_ppm("mandelbrot.ppm", image, max_iter);
211
212 free(image);
213
214 printf("Imagem salva em mandelbrot.ppm\n");
215 printf("Linhas calculadas: %d\n", rows_completed);
216 printf("Tempo total: %f segundos\n", end - start);
217 }
218
219 // =====
220 // WORKERS
221 // =====
222
223 else
224 {
225     MPI_Status status;
226     int request = 1;
227
228     while (1)
229     {
230         // Trabalhador toma a iniciativa e pede trabalho ao coordenador.
231         MPI_Send(&request,
232                 1,
233                 MPI_INT,
234                 0,
235                 TAG_REQUEST,
236                 MPI_COMM_WORLD);
237
238         Task task;
239
240         MPI_Recv(&task,
241                 sizeof(Task),
242                 MPI_BYTE,
243                 0,
244                 MPI_ANY_TAG,
245                 MPI_COMM_WORLD,
246                 &status);
247
248         if (status.MPI_TAG == TAG_STOP)
249             break;
250
251         int *buffer = malloc(sizeof(int) * task.num_rows * WIDTH);
252
253         if (buffer == NULL)
254         {
255             printf("Processo %d: erro ao alocar memoria para o buffer.\n", rank);
256             MPI_Abort(MPI_COMM_WORLD, 1);
257         }
258
259         for (int row = 0; row < task.num_rows; row++)
260         {
261             int y = task.start_row + row;
262
263             for (int x = 0; x < WIDTH; x++)
264             {
265                 double real = -2.5 + (3.5 * x / WIDTH);
266                 double imag = -1.0 + (2.0 * y / HEIGHT);
267
268                 int iter = mandelbrot(real, imag, max_iter);
269
270                 buffer[row * WIDTH + x] = iter;
271             }
272         }

```

```

273
274     MPI_Send(&task,
275              sizeof(Task),
276              MPI_BYTE,
277              0,
278              TAG_RESULT,
279              MPI_COMM_WORLD);
280
281     MPI_Send(buffer,
282              task.num_rows * WIDTH,
283              MPI_INT,
284              0,
285              TAG_RESULT,
286              MPI_COMM_WORLD);
287
288     free(buffer);
289 }
290
291
292 MPI_Finalize();
293
294 return 0;
295 }

```